THE ELECTRICITY DEMAND DISAGGREGATION CHALLENGE

Literature Review
CS2 Romande Energie x ETH

Case Study 2025/26
Master's Degree Program in Energy Science and Technology (MEST)
Energy Science Center (ESC)
Swiss Federal Institute of Technology (ETH) Zürich

Pierpaolo Musiello, Yujin Kim, Alan Matiatos, Aline Mengis, Esad Çiftçi

Coach: Kaori Fogliani

Industry Partner
Romande Energie
Filippo Tani (filippo.tani@romande-energie.ch)
Martin Péclat (martin.peclat@romande-energie.ch)
Raphael Barben (raphael.barben@romande-energie.ch)

November 25, 2025

ETH zürich

Contents

Glossary

Abstract

This document reviews various approaches to identify the most suitable method for developing an algorithm to disaggregate the load curves of Romande Energie (RE) customers. Load disaggregation allows for more accurate forecasting of electricity demand, which can lead to significant economic benefits. Poor forecasts, on the other hand, result in unwanted adjustment costs. Furthermore, understanding customer consumption patterns can help provide targeted advice on how to better manage appliance usage, for example, by avoiding the unnecessary operation of high-consumption devices during peak or high-cost periods.

The main questions that emerged from the literature are whether the proposed methods are applicable to our specific context and, if so, how well they perform in it. Accordingly, this review discusses the most relevant studies, emphasizing those that employ data similar to RE's load curves (in terms of electrical variables, sampling frequency, and statistical assumptions) and that address the disaggregation of the same types of appliances considered in our work. The analysis also highlights how the low sampling rate of our data constrains the range of applicable methods.
At the end of the review, a hybrid approach specifically designed for our case study will be proposed.

1 Introduction

In recent years, the Swiss electrical grid has been undergoing major transformations. Two key developments are driving these changes:

- The increasing number of prosumers — consumers who also store and produce electricity — due to the widespread adoption of Photovoltaic (PV) systems and batteries, making the grid increasingly unpredictable and harder to control.
- The growing electrification of households, characterized by the proliferation of high-consumption devices such as Heat Pumps (HPs), Electric Vehicles (EVs), and Air Conditioners (ACs).

RE is a Swiss company engaged in the production, distribution, and marketing of energy, and is a major supplier in French-speaking part of Switzerland. For RE, these developments have led to more frequent errors in electricity demand forecasting and, consequently, to significant economic losses. This situation has created a strong need for the company to improve the forecasting of its customers' electricity consumption.
Characterizing customer behavior by analyzing the load curves of their most used appliances has therefore become crucial for understanding the portfolio's overall appliance mix, identifying key consumption trends, and developing more accurate long-term forecasts.
However, the main challenge lies in obtaining these load curves. In practice, only the total household consumption — measured at the main electrical panel with a 15-minute sampling rate — is available, while the detailed consumption profiles of individual appliances are not directly recorded. It is therefore necessary to derive or estimate the load curves of individual appliances from the aggregate consumption data.

In [24], the concept of NILM is introduced. In short, replacing all household appliances with new ones equipped with measurement sensors would be impractical. Thus, a non-intrusive method is required: it is possible to infer the individual appliance load profiles through a disaggregation algorithm. This concept will be discussed in greater detail in Section 2.1.
The work of [24] describes methods applicable to both high-frequency and low-frequency data. However, our review focuses exclusively on the latter, since our load curves have a sampling rate of 15 minutes, corresponding to very low-frequency data. On the contrary, high-frequency algorithms operate at frequencies on the order of kilohertz, which is far beyond the temporal resolution of our dataset.

The following sections discuss several papers describing different algorithms in order to identify a suitable solution for RE: disaggregating the load curves measured at the main breaker level into the individual load curves of the appliances of interest — namely, those with the highest energy consumption.

2 Demand Disaggregation and NILM

2.1 Theoretical Background on NILM

NILM is the process of estimating the energy usage of individual appliances from aggregate power meter readings without requiring access to the individual devices or sources [9].
The core principle of NILM is based on identifying unique electrical signatures that are generated when each appliance turns on, turns off, or changes its operating state. These changes can manifest as either transient events (momentary spikes) or specific steady-state consumption patterns. NILM algorithms analyze these signatures, often referred to as load signatures, to perform energy disaggregation, which separates the total power consumption time series into the operational status and energy use of individual loads.

This approach offers significant advantages over traditional Intrusive Load Monitoring (ILM) methods, as it is more cost-effective, easier to deploy, and causes no disruption to the occupants. The ultimate goal of NILM is to provide detailed, real-time energy consumption information to encourage energy-saving behaviors among users and to enhance the efficiency of Smart Grid operations and Demand Response programs.

2.1.1 Mathematical Model

The fundamental problem of NILM can be mathematically formulated as a signal decomposition challenge. The aggregated power signal, $P(t)$, which is the input to the system, can be expressed as the sum of all individual appliance loads and a residual term representing measurement errors [9]:

$$P(t) = \sum_{i=1}^{N} P_i(t) + \epsilon(t) \tag{1}$$

where

- $P(t)$: The total measured power (input signal) at time t.
- $P_i(t)$: The power consumed by the i-th appliance at time t.
- N: The total number of appliances in the system.
- $\epsilon(t)$: A residual term representing measurement noise, unmonitored devices, or low-power continuous loads (e.g., standby power).

The core objective of a NILM algorithm is to solve this inverse problem—to estimate the individual power contributions $P_i(t)$ for all $i \in \{1, \ldots, N\}$, using only the measured aggregate signal $P(t)$. This approach is highly dependent on prior knowledge of both the appliances' load signatures and their presence in the home. Due to the general lack of

this information in practical applications, solving this decomposition problem accurately remains a significant challenge [18].

2.1.2 Steady-State and Transient Approaches

The process of disaggregating the aggregated power signal relies on analyzing distinct electrical signatures that manifest during appliance operation. These signatures are fundamentally categorized into two types based on the duration and frequency content of the electrical signal: Steady-State (low-frequency) and Transient (high-frequency) features.

1. Steady-State Load Features
   Steady-state analysis focuses on the stable electrical characteristics of an appliance when it is operating at a constant power level (e.g., fully ON or OFF). This approach is ideally suited for low-frequency data as it relies solely on the magnitude of change rather than the rapid dynamics of the signal. The primary features extracted are the changes in power consumption that define the appliance's signature. The most fundamental feature is the Power Step Change (ΔP), which represents the difference in Real Power (P) between two stable operating points. If Reactive Power (Q) is recorded, the Complex Power Vector ($\Delta P, \Delta Q$) offers a more robust signature, enabling differentiation between purely resistive loads ($\Delta Q \approx 0$) and inductive or capacitive loads [25].

2. Transient and High-Frequency Signatures
   Transient analysis examines the dynamic, short-duration changes in the power signal that occur only during the appliance's state transition (e.g., the inrush current when a motor switches on). These high-frequency characteristics provide highly unique electrical fingerprints and are essential for achieving high disaggregation accuracy, particularly in distinguishing appliances with similar steady-state consumption [25].

2.2 Overview of NILM Methodologies

NILM, which is a sophisticated signal processing and machine learning problem, relies on identifying the unique operating load signatures of each device embedded within the total signal. The primary methodologies developed to tackle this challenge can be broadly categorized into three main approaches:

Clustering-Based techniques that group similar consumption patterns, Probabilistic Models like those derived from Markov processes to model state transitions, and Deep Learning approaches which learn complex features for disaggregation. Each technique leverages different mathematical principles to effectively isolate and estimate the contribution of individual appliances.

The following three subsections will detail these critical methodologies, beginning with the distance-based clustering approach, proceeding through the probabilistic framework of Hidden Markov Models, and concluding with a brief introduction in unsupervised deep learning.

2.2.1 Clustering-Based Approach: K-means Clustering

The K-means algorithm is a clustering method in which the data is grouped into K groups or clusters, where K is the number of pre-chosen groups. This method is fundamentally a distance-based, partitioning algorithm that aims to partition n observations $(\mathbf{x} = \mathbf{x_1}, \ldots, \mathbf{x_n})$ into K clusters $(\mathbf{P_1}, \ldots, \mathbf{P_K})$. It implicitly assumes that the clusters are convex and isotropic (i.e., roughly spherical and equal in variance), relying on Euclidean distance to define similarity relative to the cluster centers $(\mathbf{c_1}, \ldots, \mathbf{c_K})$. See figure 2.1.

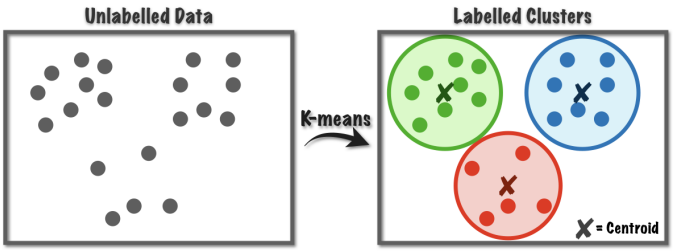


Figure 2.1: K-Means Clustering

Crucially, the effectiveness of the grouping is dependent on the quality of the underlying vector space aswell as the number of chosen clusters. Each observation $\mathbf{x}$ must be properly represented as a feature vector whose dimensions best capture the relevant characteristics (e.g., time intervals, sequences, or events) for distance-based comparisons.

1. **Conceptual Framework**
   The grouping process is an iterative centroid-based approach that seeks to minimize the Within-Cluster Sum of Squares (WCSS), also known as the inertia. This metric is defined as the sum of the squared distances between each point and the centroid of its assigned cluster:

$$Minimize \quad \sum_{i=1}^{K} \sum_{x \in P_i} \|x - c_i\|^2 \tag{2}$$

   where c_i is the centroid of cluster P_i. The algorithm converges when the centroids stabilize, or the reduction in WCSS becomes negligible. The initial selection of the K centroid positions is a critical factor influencing the final cluster configuration.

2. **Algorithm**

   - Assignment Step (Cluster Formation): Assign each data point x to the closest prototype vector c_i. P_i is then the cluster containing the objects x which are closest to c_i.
   - Update Step (Centroid Recalculation): Update the new centroids until convergence or allocated time ends according to [6]:

$$c_i = \frac{1}{|P_i|} \sum_{x \in P_i} x, \quad \forall i \in [[1, K]] \tag{3}$$

3. **Mini-Batch optimization for K-means clustering**
   Since K-means clustering has high computational costs and our dataset is substantial in size, it is useful to adopt an optimization specifically designed for large-scale scenarios. [19] proposes the Mini-Batch K-means algorithm, which updates the centroids using small, randomly sampled batches (mini-batches) instead of the full dataset. This significantly reduces computational time and memory usage while maintaining clustering quality close to that of standard K-means. The method performs incremental centroid updates based on each mini-batch, enabling efficient processing even for web-scale datasets. While this approach is faster in reaching convergency, the results are slightly less accurate than standard K-means.

2.2.2 Probabilistic Model Approach: Hidden Markov Models (HMM)

Hidden Markov Models (HMM) is built upon an extension of the Markov chain. A Markov chain is a probabilistic model that describes sequences of random variables, or states, where each state can take on values from a certain set — such as words, tags, or symbols representing things like weather conditions. The key assumption of a Markov chain is that the future depends only on the present state, not on the sequence of past states. In other words, previous states influence the future only through their effect on the current state. It is like predicting tomorrow's weather based solely on today's weather, without considering

what the weather was like yesterday [11].

While a standard Markov chain's states (like "sunny," "cloudy," or "rainy" weather) are directly observable, an HMM assumes that the sequence of states you are truly interested in—the hidden states—is not directly visible. Instead, these hidden states indirectly influence a sequence of observable emissions (or observations). HMM is a tool for representing probability distributions over sequences of observations [8].

1. **Conceptual Framework**
   An observation X_t at time t is produced by a stochastic process, but the state Z_t of this process cannot be directly observed, i.e. it is *hidden*. [17] This hidden process is assumed to follow the Markov property, meaning that the current state Z_t at time t depends only on the immediately preceding state Z_{t-1}. Such a process is referred to as a *first-order Markov model*. In general, an nth-order Markov model considers dependencies on the n previous states [5].

2. **Components**
   The model probabilistically links the two layers — the hidden process and the observed outcome. The key components essential for the complete operation of the HMM are:

   - **States (Q):** A set of N states, $Q = \{q_1, q_2, \ldots, q_N\}$. These are the hidden events.
   - **Transition Probability Matrix ($\mathbf{A}$):** A matrix $(\mathbf{A}) = [a_{ij}]$, where each entry a_{ij} represents the probability of moving from state i to state j.
     - Constraint: $\sum_{j=1}^{N} a_{ij} = 1$ for all i.
   - **Observation Likelihoods/Emission Probabilities ($\mathbf{B}$):** A set of probabilities $(\mathbf{B}) = \{b_i(o_t)\}$, where $b_i(o_t)$ expresses the probability of an observation o_t (e.g., a word from a vocabulary V) being generated from a state q_i.
   - **Initial Probability Distribution (π):** A vector $\pi = \{\pi_1, \pi_2, \ldots, \pi_N\}$, where π_i is the probability that the Markov chain will start in state i.
     - Constraint: $\sum_{i=1}^{N} \pi_i = 1$.

3. **Three Problems of Interest**
   An influential tutorial by Lawrence R. Rabine [17], which was derived from tutorials by Jack Ferguson in the 1960s, introduced the idea that hidden Markov models should be defined by three fundamental problems:

   - **Problem 1 (Likelihood):** Given an observation sequence $X_1, X_2, \ldots, X_N$ and a HMM $\lambda = (A, B, \pi)$, how do we efficiently compute the probability of the observations given the model, $P(X_{1:N} \mid \lambda)$?

- **Problem 2 (Decoding):** Given observations $X_1, \ldots, X_N$ and the model λ, how do we find the "correct" hidden state sequence $Z_1, \ldots, Z_N$ that best "explains" the observations?
- **Problem 3 (Learning):** How do we adjust the model parameters $\lambda = (A, B, \pi)$ to maximize the probability $P(X_{1:N} \mid \lambda)$?

4. **Algorithms**
   The practical utility of the HMM stems from the existence of specific algorithms that can efficiently solve the three fundamental problems outlined above. While exhaustively searching through all possible hidden state sequences would result in an exponential computational complexity, these algorithms utilize the principle of Dynamic Programming to solve the problems in near-linear time complexity.

   - **The Forward-Backward Algorithm:** It uses Dynamic Programming, an approach where a complex problem is broken down into simple, overlapping subproblems. Instead of checking every single possible hidden state path (which would be extremely slow), it builds up the total probability by reusing intermediate calculations in a structure called a trellis, significantly reducing the time complexity.
   - **The Viterbi Algorithm:** It's essentially a pathfinding algorithm. It travels through the HMM's trellis structure, but at each step, instead of summing probabilities (like the Forward Algorithm), it only keeps track of the maximum probability path leading to the current state.
   - **The Baum-Welch Algorithm:** It is the Expectation-Maximization (EM) method for training HMMs, which iteratively uses the Forward-Backward Algorithm to calculate expected counts (E-Step) and then uses these counts to re-estimate the model parameters ($\mathbf{A}$ and $\mathbf{B}$) (M-Step) until convergence. [11]

5. **Application of HMMs to NILM and Case Study Limitations**
   In the context of using HMMs for NILM, as for example shown by Jordan Holweger et al. [10] the model components are defined as:

   - **Hidden States (Z_t):** The operational state of a target appliance (e.g., $Q = \{q_1 : \text{'Off'}, q_2 : \text{'On'}\}$).
   - **Observations (X_t):** The aggregate power reading from the smart meter at time t (i.e., each 15-minute interval).
   - **The Decoding Problem (Problem 2):** This is the core task of NILM. The Viterbi algorithm is used to find the most probable sequence of hidden states (appliance activity) that best explains the observed aggregate power data.
   - **The Learning Problem (Problem 3):** As the data used in this study is unlabeled, the Baum-Welch algorithm would be required to attempt to learn the model parameters $(\mathbf{A}, \mathbf{B}, \pi)$ directly from the aggregate signal.

Despite this theoretical alignment, the HMM methodology is unsuitable for this study due to the low resolution of the data. The performance of HMMs is known to degrade significantly as data frequency decreases. These models rely on detecting the sharp, instantaneous events of state transitions (an appliance turning 'On' or 'Off') to accurately estimate the transition probabilities ($\mathbf{A}$). At a 15-minute resolution, these events are averaged and over one or more intervals, rendering them indistinct. This loss of signal fidelity makes it difficult for the learning algorithm to converge on an accurate model [10].

2.2.3 Unsupervised Deep Learning Approaches

Recent advances have introduced Deep Learning techniques to NILM, which are capable of learning directly from the complex $\mathbf{P}_{total}$ time-series data without the need for manual feature engineering. For unsupervised NILM, **Autoencoders (Autoencoder (AE))** are a primary-used architecture.

1. **The AE as an Unsupervised learning Algorithm**
   An AE is a neural network trained on a simple, yet powerful, unsupervised task: it is given an input x and is trained to **reconstruct that same input** as its output ($y^{(i)} = x^{(i)}$), thereby learning an approximation of the identity function ($h_{W,b}(x) \approx x$). The model is composed of two parts: an **Encoder** that compresses the input into a lower-dimensional "latent space," and a **Decoder** that tries to rebuild the original input from that compressed representation.

   To prevent the network from learning the "trivial" identity (i.e., simply copying the input), it is forced to learn a compressed, useful representation of the data. This is achieved by applying a constraint, most commonly:

   - **Dimensionality Constraint:** The network is designed with a "bottleneck," a hidden layer with fewer neurons than the input layer. This forces the AE to learn the most important, structural features of the data to successfully pass it through this bottleneck and reconstruct it.
   - **Sparsity Constraint:** An alternative approach that adds a penalty term to the objective function. This term forces the average activation of the j-th hidden unit, $\hat{\rho}_j$, to be close to a small sparsity parameter, ρ (e.g., 0.05). This penalty is a **Kullback-Leibler (KL) divergence**, which measures the difference between the two distributions:

     $$\sum_j \mathrm{KL}(\rho||\hat{\rho}_j) = \sum_j \left[\rho \log \frac{\rho}{\hat{\rho}_j} + (1-\rho) \log \frac{1-\rho}{1-\hat{\rho}_j}\right]$$

     Minimizing this function penalizes hidden units for activating too often, forcing the network to learn a sparse representation.

By learning to reconstruct the input from this constrained (either low-dimensional or sparse) encoding, the AE effectively discovers the underlying structure of the data [21]. In the context of NILM, it learns to represent a complex, 96-sample daily load curve as a much smaller, dense vector of features.

2. **Sequence-to-Sequence (Seq2Seq) AEs for Disaggregation**
   A standard AE is not designed for sequential time-series data. This led to the adoption of **Sequence-to-Sequence (Seq2Seq)** models.

   In the supervised domain, various Deep Neural Networks (DNN) models have been applied to NILM, categorized by their input-output shape: mapping an input sequence of aggregate power to an output sequence of the **same length** (sequence-to-sequence), a **shorter** sequence (sequence-to-subsequence), or a **single power estimate** (sequence-to-point) [13]. For **unsupervised NILM**, the Seq2Seq AE is cleverly adapted. The network is not trained to reconstruct its own input. Instead, the model is designed to:

   - **Input (Encoder):** Take a sequence of the aggregate load, $\mathbf{P}_{total}$.
   - **Output (Decoder):** Produce *multiple* output sequences, one for each target appliance (e.g., $P_{appliance1}$, $P_{appliance2}$,... P_{other}).

   The model is then trained **without appliance-level labels**. The network's objective is to ensure that the **sum** of all its output appliance sequences is as close as possible to the original aggregate $\mathbf{P}_{total}$ input. By minimizing this "reconstruction error," the network is forced to learn how to separate the aggregate signal into its constituent parts, effectively learning the individual appliance signatures from the mix.

3 Classification Methods

In NILM, classifiers are needed to identify which appliances contribute to the total household power signal. The aggregate load curve is a composite of multiple overlapping appliance signatures. Classification algorithms separate these mixed patterns by assigning portions of the signal to appliance categories based on extracted features, such as statistical features like mean power and variance, temporal features like time of day, or external features like weather data.

These algorithms differ from clustering, which groups data based on feature similarity but does not provide an inherent mapping to a specific appliance. Classifiers, when trained on data with known labels, learn this formal mapping. This allows them to directly assign an appliance label (e.g., "Heat Pump") to a set of features, rather than just grouping them as "Cluster A".

In the following sections aswell as in the figure 3.1, two well-known classifiers are described in more detail.

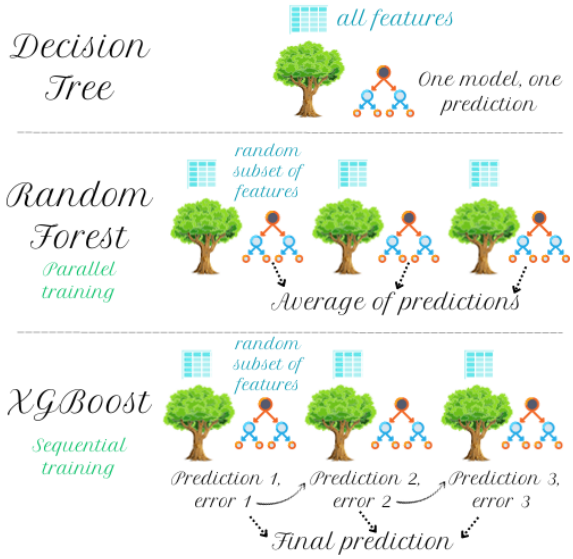


Figure 3.1: Classifiers Comparison

3.1 Random Forest

Random Forest is a machine learning model that makes predictions by combining multiple decision trees. Each tree gives its own prediction, and the forest aggregates these results using either the **majority vote** (for classification, e.g., appliance recognition) or the **average** (for regression, e.g., consumption estimation).
The mathematical principles of RF are summarized below, following the formulation presented in [2].

3.1.1 Training phase

Given a labeled dataset:

$$D = \{(x_i, y_i)\}_{i=1}^{N} \tag{4}$$

where N is the total number of samples, x_i is a p-dimensional feature vector, and y_i is the corresponding ground-truth label.

The predictive power of a Random Forest comes from building an ensemble of decorrelated decision trees, $\{T_1, T_2, ..., T_B\}$, using two distinct layers of randomization.

The first layer is bootstrap aggregating (bagging), which applies to the data. Each of the B trees is trained on a unique bootstrap sample, D_b, created by drawing N samples with replacement from the original dataset D. This means any given tree sees a slightly different version of the training data, preventing the trees from being identical.

The second layer of randomization applies to the features. As each tree is built, it must make a series of splits to partition the data into smaller, more homogeneous groups. At each node, a standard decision tree would test all p available features to find the optimal split. A Random Forest, however, restricts this choice, allowing the tree to search from only a random subset of m features (where $m < p$). The algorithm selects the best split from this random subset based on an impurity measure, which evaluates how well the split separates the classes into "pure" child nodes. The most common impurity metric used is the Gini index, which the algorithm aims to minimize:

$$G(t, j) = \sum_{k \in \{L,R\}} \frac{N_k}{N} \sum_{c} p_{k,c}(1 - p_{k,c})$$

where L and R are the left and right child nodes after a split, N_k is how many samples are in node k, $p_{k,c}$ is the proportion of class c in child node k.
The split with the smallest $G(t, j)$ is selected.

This combination of bootstrap sampling and random feature selection ensures that the trees are decorrelated, improving the ensemble's overall generalization and stability.

3.1.2 Prediction Phase

For a new sample x, each tree T_B predicts a class label $T_b(x)$.
The forest outputs either the majority vote across all trees (classification):

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \ldots, T_B(x)\}.$$

or the average of all tree predictions (regression):

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} T_b(x).$$

By combining the outputs of many diverse trees—which were decorrelated during training via bootstrap sampling and random feature selection—the ensemble effectively averages out the errors of individual, high-variance decision trees. This process allows the final model to achieve low bias while significantly reducing variance, resulting in strong generalization and robustness to noise.

3.1.3 Random Forest in NILM

RF, one of the most famous classifiers, is particularly suitable for NILM. Several studies have demonstrated its effectiveness, such as [14] and [23]. These works highlight several reasons why RF performs well in this context:

1. Robustness to irrelevant features: NILM feature sets often contain redundant or weakly informative variables. RF naturally down-weights such features because each split considers only a random subset of variables.

2. Adaptability: In the adaptive NILM model proposed in [14], RF is integrated with an unknown pattern recognition and online update module. It learns incrementally from new or previously unseen appliance signatures, improving identification over time—from 78.47 % to 94.17 % accuracy in their case study.

3. Simplicity of calibration: Only a few parameters (number of trees, number of features, and minimum node size) control performance. In NILM, where data come from diverse appliances, this simplicity allows efficient training and re-training as new load types appear.

4. Stability and accuracy: RF overcomes the instability of single decision trees, which are sensitive to small changes in data. By combining many decorrelated trees, RF delivers stable and accurate classification of mixed appliance signals.

5. Performance: the experiments of [23] achieved classification accuracy ≈ 0.97 and F-score ≈ 0.98, outperforming k-NN and neural-network-based multi-label classifiers on both real and public NILM datasets.

3.2 XGBoost

XGBoost (Extreme Gradient Boosting) is a tree-based ensemble learning algorithm that improves prediction accuracy by combining many weak learners (decision trees) through a process called boosting.

Unlike Random Forest (RF), which builds trees independently and in parallel using bagging, XGBoost constructs trees sequentially, where each new tree focuses on correcting the errors made by the previous ones.

3.2.1 Operation Principles

The operational principles of XGBoost, as introduced by [4], are a sequential evolution of gradient boosting. The model is trained in an additive manner, where the prediction for a sample x_i is the sum of K individual regression trees:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F},$$

where each f_k is a tree in the function space $\mathcal{F}$.
Unlike Random Forest, where trees are built independently, XGBoost trains trees sequentially to correct the errors of the preceding ones. This is achieved by minimizing a regularized objective function that balances prediction error (the loss term L) with model complexity (the regularization term Ω):

$$\text{Obj} = \sum_{i=1}^{N} L(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

The regularization term penalizes complexity using $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$, which controls the number of leaves T (via γ) and the magnitude of the leaf scores w (via $L2$ regularization λ). To find the optimal tree f_t to add at each iteration t, the algorithm minimizes the objective. The key innovation of XGBoost is to approximate this objective using a **second-order Taylor expansion**, which transforms the complex functional optimization into a solvable numeric one:

$$\text{Obj}^{(t)} \approx \sum_{i=1}^{N} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Here, g_i and h_i are the gradient and hessian (the first and second derivatives) of the loss function, evaluated at the previous step's prediction:

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \quad \text{and} \quad h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$$

This use of second-order information (the hessian, h_i) allows the algorithm to find a more precise path to the minimum, enabling faster convergence and superior accuracy compared to traditional gradient boosting methods that rely only on the first derivative.

3.2.2 XGBoost in NILM

According to [20], XGBoost performs exceptionally well in NILM. In their study, the authors compared several machine learning classifiers — including Random Forest, k-NN, Decision Trees, and XGBoost — on both real and public datasets. The results clearly demonstrate the superiority of XGBoost in terms of predictive performance and robustness.
The main findings regarding XGBoost can be summarized as follows:

1. **High accuracy and robustness:** XGBoost achieved the highest classification accuracy among all tested models, as well as the lowest mean absolute error (MAE) and root mean square error (RMSE).

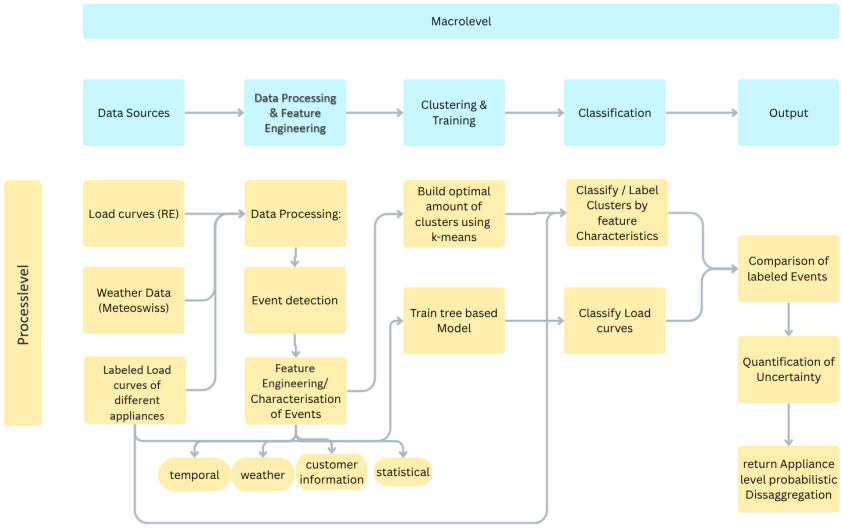
2. **Strong generalization:** Due to its gradient boosting mechanism and regularization, XGBoost effectively handled noisy and imbalanced NILM data, maintaining high performance across different load types.

3. **Efficiency:** The algorithm demonstrated fast training and prediction times while preserving high accuracy, making it suitable for large-scale NILM applications.

4 Modelling Approach

In practice, both unsupervised HMMs and autoencoder-based deep learning models face significant challenges when applied to aggregated active power data with a 15-minute sampling interval. The coarse temporal resolution blurs appliance-level transitions, making it difficult for HMMs to capture meaningful state changes and for autoencoders to learn discriminative features. Additionally, using only active power as the input limits the information available for modeling, causing autoencoders to reconstruct mainly aggregated trends rather than disentangling individual appliance contributions.

To address the challenge of disaggregating unlabeled 15-minute resolution load curves, this project employs a hybrid modeling approach, as detailed in the process-level flowchart. This methodology leverages the complementary strengths of unsupervised and supervised machine learning techniques, specifically designed to overcome the lack of initial ground-truth labels while building towards an accurate, probabilistic disaggregation output. The following sub-chapters detail each step of this process.



4.1 Data Sources

- `Load curves (RE):`

  The primary, unlabeled consumption data used in this study is provided by RE. It consists of load curves recorded from a set of smart meters installed at the interface between customers (e.g., buildings) and the power grid. The data represents the energy flow to and from customers, sampled at 15-minute intervals throughout the year 2024.

  – **Version 1.** The first version of the dataset includes, for each record, the smart meter ID, the flow type, and the corresponding energy load in kWh. The

measurements indicate the total energy exchanged during each 15-minute interval. Two types of flows are distinguished: `consumption (cons)`, representing electricity injected into the building from the grid, and `production (prod)`, representing electricity fed back to the grid from local generation (typically photovoltaic systems).

Since the meters are placed at the grid interface, the measurements reflect only the net energy exchange between the building and the grid. Consequently, for customers equipped with photovoltaic (PV) installations, both consumption and production curves may be present. However, these curves do not directly correspond to the true internal consumption or generation values, as self-consumed PV energy within the building is not captured by the meters.

- **Version 2 (planned extension).** A future version of the dataset will incorporate additional contextual information to enhance interpretability. Specifically, it will include:
  - **Customer type**, distinguishing between residential (single- or multi-family households) and commercial users.
  - **Matched production and consumption curves** sharing the same customer ID, allowing for direct correspondence between the two flow types.

- `Weather Data (Meteoswiss):`
  Since heating and cooling demand strongly depend on outdoor temperature, and PV production is influenced by solar radiation, incorporating weather data into the analysis can improve algorithmic performance. Previous studies, such as [16], have demonstrated that including weather-related features can enhance clustering algorithms, particularly for AC load detection. Similar findings are reported for HP detection in Switzerland by [22].

  For this study, MeteoSwiss weather data will be used [15]. Within the RE supply area, nine weather stations are available. The average of all stations is considered for the analysis, as done in [22], with potential weighting based on customer density. The used weather stations are the following:

  1. Nyon/Changins
  2. Bière
  3. Vevey/Corseaux
  4. St. Prex
  5. Mathod
  6. Bullet/ La Frétaz
  7. Villars-Tiercelin

8. Pully
9. La Dôle

- `Labeled Load curves:`
  In previous studies, open-source labeled datasets have been used for NILM. For example, [3] employs the ECO and REFIT datasets, which contain labeled load curves collected at high frequency in Switzerland and the UK. This labeled data were then used for NILM in low frequency smart meter data from Switzerland. Even though the labeled data includes only load curves from appliances different from those considered in this case study, it has been shown that the choice of dataset significantly affects the accuracy of the results.

In this study, typical load curves of individual appliances will only be used as reference data. Since these curves originate from literature and do not exactly represent the environmental conditions and appliance types in our context, they will be used solely for feature identification and data preprocessing, and not directly for model training.

4.2 Data Processing & Feature Engineering:

An important step is to prepare the provided data into a usable format. To summarize, the model ingests three primary data sources, which have been described in detail in the previous section 4.1.

Given the large size of our dataset (over 52 million rows) and that the smart meters were originally designed for billing rather than analysis, data cleaning and preprocessing are essential. As shown in [12], resampling and cleaning methods significantly improve the accuracy of energy disaggregation. In particular, data resampling helps handle missing or zero readings, filter out erroneous measurements, and identify gaps in the time series, which are crucial steps for improving model performance. The following preprocessing steps are based on established approaches discussed in [7].

Normalization is a crucial preprocessing step in NILM, as electrical signals from different appliances can vary by several orders of magnitude. Normalization further stabilizes and accelerates model training and improves the comparability of datasets collected from different sources [7, 1].
Common normalization techniques in the energy domain include:

1. **Z-score normalization**: Rescales data to have zero mean and unit standard deviation:
$$x' = \frac{x - \mu}{\sigma}$$
   Suitable for approximately Gaussian data or when outliers are present, often used in machine learning models such as neural networks.

2. **Min-Max normalization**: Transforms data to a fixed range, typically [0,1]:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

   Useful for non-Gaussian data with zeros and spikes, commonly applied in distance-based models such as KNN and clustering algorithms.

Afterwards an `Event Detection` algorithm is applied to identify significant power changes (i.e., appliance state transitions) within the time series. These detected events are subsequently characterized through various features. Importantly, both the unlabeled load curves from RE and the external labeled datasets are characterized using the exact same feature extraction logic. This ensures a common feature space, where the features are categorized as:

- Temporal (e.g., time of day, day of week)
- Weather (e.g., temperature, irradiance)
- Customer information
- Statistical (e.g., power step ΔP, event duration)

4.3 Hybrid Disaggregation Framework

After `Feature Engineering`, the model splits into two parallel analytical paths to generate appliance labels for the RE events.

4.3.1 Path A: Unsupervised Clustering and Classification

This path uses an unsupervised approach to find the dominant, naturally occurring patterns in the utility's data.

1. Clustering: The feature set from the unlabeled RE events is fed into a **Mini-Batch K-means** algorithm to `Build an optimal amount of clusters`.
2. Labeling: The resulting clusters are then analyzed in the `Classify / Label Clusters` step. This module assigns a probable appliance label (e.g., "Heat Pump", "EV") to each cluster by comparing its feature characteristics (e.g., its centroid) against the known feature distributions from the external labeled appliance dataset.

4.3.2 Path B: Supervised Classification (Tree-based)

This path employs a supervised model to classify each event directly.

1. Training: A `Train tree based Model` (such as the Random Forest or XGBoost models discussed in Chapter 3) is trained. This model learns the mapping from features to appliance labels using the pre-processed external **labeled load curves**.

2. Classification: This trained classifier is then applied to the entire feature set of unlabeled RE events to `Classify Load curves`, assigning an appliance label to each event based on its learned feature patterns.

4.4 Probabilistic Output and Uncertainty

The outputs from both paths—the cluster-based labels (Path A) and the classifier-based labels (Path B)—are fed into a `Comparison of labeled Events`.

- Uncertainty: The agreement or disagreement between the two methods for any given event is used for the `Quantification of Uncertainty`. For example, an event labeled "Heat Pump" by the clustering path but "EV" by the tree-based classifier will be flagged with high uncertainty.
- Final Output: This comparison allows the model to `return Appliance level probabilistic Disaggregation`. Instead of a single, deterministic output, the model will provide a probabilistic estimate of appliance-specific consumption, informed by the consensus (or lack thereof) between the two methodologies.

5 Conclusion

This literature review has analyzed various methodologies for Non-Intrusive Load Monitoring (NILM) to address RE's challenge of electricity demand disaggregation. One of the main finding is that the 15-minute sampling frequency of the RE dataset severely constrains the applicable techniques, definitively rendering high-frequency transient-based methods unusable. Therefore, the emphasis should be placed on algorithms capable of interpreting steady-state power variations, such as the clustering and classification methods discussed.

The review of these methods highlighted their respective strengths and weaknesses, particularly in the context of unlabeled data. A purely unsupervised approach like Mini-Batch K-means clustering may successfully identify patterns but struggles with accurate appliance labeling. Conversely, a purely supervised classifier, while robust (e.g., Random Forest, XGBoost), requires a large, representative, and accurately labeled training set, which is not available for the RE customer base.

Therefore, the hybrid modeling approach detailed in Chapter 4 is proposed. This framework is a pragmatic solution designed to leverage the strengths of both paradigms. The unsupervised clustering path (Path A) will identify the dominant, naturally-occurring consumption patterns within the RE dataset, while the supervised classification path (Path B), trained on external labeled data, will provide a strong, feature-based baseline for appliance identification.
The advantage of this approach lies in the following comparison of the two paths, which enables a quantification of uncertainty. This moves the final output beyond a simple deterministic label to a more realistic and actionable probabilistic disaggregation.

In conclusion, while the literature confirms the significant difficulty of low-frequency NILM, it also provides the necessary building blocks for a viable solution. The proposed hybrid model represents a robust and data-driven path forward. The immediate next steps will involve the intensive data preprocessing and feature engineering outlined in the model, followed by the clustering and classifying steps. Successful execution will provide RE with a scalable tool to characterize their customers' consumption, identify the impact of key loads such as heat pumps and electric vehicles, and ultimately improve demand forecasting.

[1] Georgios-Fotios Angelis et al. “NILM Applications: Literature Review of Learning Approaches, Recent Developments and Challenges”. In: *Energy and Buildings* 261 (2022). DOI: 10.1016/j.enbuild.2022.112065.

[2] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[3] Arnab Chatterjee and Philipp Heer. “Non-Intrusive Load Monitoring (NILM) with very low-frequency data from smart meters in Switzerland”. In: *Energy and Buildings* 344 (2025), p. 116002. ISSN: 0378-7788. DOI: https://doi.org/10.1016/j.enbuild.2025.116002. URL: https://www.sciencedirect.com/science/article/pii/S0378778825007327.

[4] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’16)*. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785.

[5] Alperen Degirmenci. *Introduction to Hidden Markov Models*. Tech. rep. Excerpt from project report for MIT 6.867 Machine Learning class (Fall 2014). Harvard University, School of Engineering and Applied Sciences, 2014. URL: https://projects.iq.harvard.edu/.

[6] ETH Zürich, Signal and Information Processing Laboratory. *SV7: K-means and Spectral Clustering*. Tech. rep. Fachpraktikum Signalverarbeitung Lab Material. ETH Zürich.

[7] Cheng Fan et al. “A Review on Data Preprocessing Techniques Toward Efficient and Reliable Knowledge Discovery From Building Operational Data”. In: *Frontiers in Energy Research* 9 (2021), p. 652801. DOI: 10.3389/fenrg.2021.652801.

[8] Zoubin Ghahramani. “An Introduction to Hidden Markov Models and Bayesian Networks”. In: *International Journal of Pattern Recognition and Artificial Intelligence* 15.1 (2001), pp. 9–42. ISSN: 0218-0014. DOI: 10.1142/S0218001401000836. URL: https://doi.org/10.1142/S0218001401000836.

[9] George W. Hart. “Nonintrusive Appliance Load Monitoring”. In: *Proceedings of the IEEE* 80.12 (1992), p. 1870. DOI: 10.1109/5.192069.

[10] Jordan Holweger et al. “Unsupervised algorithm for disaggregating low-sampling-rate electricity consumption of households”. In: *Sustainable Energy, Grids and Networks* 19 (2019), p. 100244.

[11] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Draft of August 24, 2025. Draft Edition, 2025. Chap. Appendix A: Hidden Markov Models. URL: https://web.stanford.edu/~jurafsky/slp3/.

[12] Maria Kaselimi et al. “Towards Trustworthy Energy Disaggregation: A Review of Challenges, Methods, and Perspectives for Non-Intrusive Load Monitoring”. In: *Sensors* 22.15 (2022), p. 5872. DOI: 10.3390/s22155872.

[13] Wenpeng Luan et al. “Leveraging sequence-to-sequence learning for online non-intrusive load monitoring in edge device”. In: *International Journal of Electrical Power and Energy Systems* 148 (2023), p. 108910. DOI: 10.1016/j.ijepes.2022.108910.

[14] Jie Mei et al. “Random Forest Based Adaptive Non-Intrusive Load Identification”. In: *Proceedings of the 2014 International Joint Conference on Neural Networks (IJCNN)*. 2014, pp. 1978–1983. DOI: 10.1109/IJCNN.2014.6889897.

[15] MeteoSchweiz. *Messwerte und Messnetze: Messwerte Lufttemperatur 10 min*. Accessed: 2025-10-28. 2025. URL: https://www.meteoschweiz.admin.ch/service-und-publikationen/applikationen/messwerte-und-messnetze.html#param=messwerte-lufttemperatur-10min&table=false.

[16] Rodrigo Porteiro and Sergio Nesmachnow. “Detecting Air Conditioning Usage in Households Using Unsupervised Machine Learning on Smart Meter Data”. In: *Smart Cities*. Ed. by Sergio Nesmachnow and Luis Hernández Callejo. Cham: Springer Nature Switzerland, 2023, pp. 233–247. ISBN: 978-3-031-28454-0.

[17] Lawrence R. Rabiner. “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition”. In: *Proceedings of the IEEE* 77.2 (1989), pp. 257–286. ISSN: 0018-9219. DOI: 10.1109/5.18626. URL: https://doi.org/10.1109/5.18626.

[18] Hasan Rafiq et al. “A review of current methods and challenges of advanced deep learning-based non-intrusive load monitoring (NILM) in residential context”. In: *Energy & Buildings* 305 (2024), p. 113890. DOI: 10.1016/j.enbuild.2024.113890.

[19] David Sculley. “Web-Scale K-Means Clustering”. In: *Proceedings of the 19th International Conference on World Wide Web*. 2010, pp. 1177–1178. DOI: 10.1145/1772690.1772862.

[20] Muhammad Shabbir et al. “Comparative Analysis of Machine Learning Techniques for Non-Intrusive Load Monitoring”. In: *Electronics* 13.8 (2024), p. 1420. DOI: 10.3390/electronics13081420.

[21] The Stanford UFLDL Team. *Autoencoders*. Unsupervised Feature Learning and Deep Learning Tutorial. 2014. URL: http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/ (visited on 10/29/2025).

[22] Andreas Weigert et al. “Detection of heat pumps from smart meter and open data”. In: *Energy Informatics* 3.1 (2020), p. 21. ISSN: 2520-8942. DOI: 10.1186/s42162-020-00124-6. URL: https://doi.org/10.1186/s42162-020-00124-6.

[23] Xin Wu, Yuchen Gao, and Dian Jiao. "Multi-Label Classification Based on Random Forest Algorithm for Non-Intrusive Load Monitoring System". In: *Processes* 7.6 (2019), p. 337. DOI: 10.3390/pr7060337.

[24] Misha Zeifman and Keren Roth. "Nonintrusive Appliance Load Monitoring: Review and Outlook". In: *IEEE Transactions on Consumer Electronics* (2011). DOI: 10.1109/TCE.2011.5735484.

[25] Ahmed Zoha et al. "Non-Intrusive Load Monitoring Approaches for Disaggregated Energy Sensing: A Survey". In: *Sensors* 12 (12 2012), pp. 16838–16866. DOI: 10.3390/s121216838.